

Under the Hood of Compile-Time Instrumentation for Go

OTel Night Berlin v2026.09 · 24 September 2026

Kemal Akkoyun · Maintainer, OpenTelemetry Go Compile-Time Instrumentation · Datadog

\$whoami

- Software engineer at Datadog (APM, Go)
- Maintainer, OpenTelemetry Go Compile-Time Instrumentation
- Prometheus Steering Committee · `client_golang` maintainer
- Emeritus: Thanos, Parca
- @kakkoyun · kakkoyun.me

What makes Go great?

Simplicity



Programs must be written for people to read,
and only incidentally for machines to execute.

— Hal Abelson

- Simple to understand.
- Simple to troubleshoot.
- Simple to operate.

No magic



Magic seems great while coding...



**...but debugging magical
code is 10x harder**



There's always a "but"...



Observability signals

Signal	Answers
Logs	What happened?
Metrics	How much and how fast?
Traces	Where did the time go? (fancy logs)
Profiles	Where did the CPU go? (<i>Alpha</i>)

The manual way

Let's see how manual instrumentation looks...

Manual instrumentation: Go 🐭 setup (simplified)

```
var tracer trace.Tracer

func main() {
    ctx := context.Background()
    exp, _ := newExporter(ctx)

    // Create tracer provider with batch span processor
    tp := newTraceProvider(exp)
    defer func() { _ = tp.Shutdown(ctx) }()

    otel.SetTracerProvider(tp)

    // Finally, set the tracer
    tracer = tp.Tracer("ExampleService")
}
```

Go 🐹 setup (realistic)

```
package main

import (
    "context"
    "errors"
    "time"

    "go.opentelemetry.io/otel"
    "go.opentelemetry.io/otel/exporters/stdout/stdoutlog"
    "go.opentelemetry.io/otel/exporters/stdout/stdoutmetric"
    "go.opentelemetry.io/otel/exporters/stdout/stdouttrace"
    "go.opentelemetry.io/otel/log/global"
    "go.opentelemetry.io/otel/propagation"
    "go.opentelemetry.io/otel/sdk/log"
    "go.opentelemetry.io/otel/sdk/metric"
    "go.opentelemetry.io/otel/sdk/trace"
)

// setupOTelSDK bootstraps the OpenTelemetry pipeline.
// If it does not return an error, make sure to call shutdown for proper cleanup.
func setupOTelSDK(ctx context.Context) (shutdown func(context.Context) error, err error) {
    var shutdownFuncs []func(context.Context) error

    // shutdown calls cleanup functions registered via shutdownFuncs.
    // The errors from the calls are joined.
    // Each registered cleanup will be invoked once.
    shutdown = func(ctx context.Context) error {
        var err error
        for _, fn := range shutdownFuncs {
            err = errors.Join(err, fn(ctx))
        }
        shutdownFuncs = nil
        return err
    }

    // handleError calls shutdown for cleanup and makes sure that all errors are returned.
    handleError := func(inErr error) {
        err = errors.Join(inErr, shutdown(ctx))
    }

    // Set up propagator.
    prop := newPropagator()
    otel.SetTextMapPropagator(prop)
```

```
    // Set up trace provider.
    tracerProvider, err := newTracerProvider()
    if err != nil {
        handleError(err)
        return
    }
    shutdownFuncs = append(shutdownFuncs, tracerProvider.Shutdown)
    otel.SetTracerProvider(tracerProvider)

    // Set up meter provider.
    meterProvider, err := newMeterProvider()
    if err != nil {
        handleError(err)
        return
    }
    shutdownFuncs = append(shutdownFuncs, meterProvider.Shutdown)
    otel.SetMeterProvider(meterProvider)

    // Set up logger provider.
    loggerProvider, err := newLoggerProvider()
    if err != nil {
        handleError(err)
        return
    }
    shutdownFuncs = append(shutdownFuncs, loggerProvider.Shutdown)
    global.SetLoggerProvider(loggerProvider)

    return
}

func newPropagator() propagation.TextMapPropagator {
    return propagation.NewCompositeTextMapPropagator(
        propagation.TraceContext{},
        propagation.Baggage{},
    )
}
```

```
func newTracerProvider() (*trace.TracerProvider, error) {
    traceExporter, err := stdouttrace.New(
        stdouttrace.WithPrettyPrint())
    if err != nil {
        return nil, err
    }

    tracerProvider := trace.NewTracerProvider(
        trace.WithBatcher(traceExporter,
            // Default is 5s. Set to 1s for demonstrative purposes.
            trace.WithBatchTimeout(time.Second)),
    )
    return tracerProvider, nil
}

func newMeterProvider() (*metric.MeterProvider, error) {
    metricExporter, err := stdoutmetric.New()
    if err != nil {
        return nil, err
    }

    meterProvider := metric.NewMeterProvider(
        metric.WithReader(metric.NewPeriodicReader(metricExporter,
            // Default is 1m. Set to 3s for demonstrative purposes.
            metric.WithInterval(3*time.Second))),
    )
    return meterProvider, nil
}

func newLoggerProvider() (*log.LoggerProvider, error) {
    logExporter, err := stdoutlog.New()
    if err != nil {
        return nil, err
    }

    loggerProvider := log.NewLoggerProvider(
        log.WithProcessor(log.NewBatchProcessor(logExporter)),
    )
    return loggerProvider, nil
}
```

Every function

```
func httpHandler(w http.ResponseWriter, r *http.Request) {  
    ctx, span := tracer.Start(r.Context(), "my_span")  
    defer span.End()  
  
    // YOUR BUSINESS LOGIC GOES HERE  
}
```

Manual: Python 🐍

```
from opentelemetry import trace
from opentelemetry.trace import Status, StatusCode

def process_order(order_id):
    tracer = trace.get_tracer(__name__)
    with tracer.start_as_current_span("process-order") as span:
        span.set_attribute("order.id", order_id)

        try:
            # Your business logic here
            validate_order(order_id)
            charge_payment(order_id)
            ship_order(order_id)
        except Exception as e:
            span.record_exception(e)
            span.set_status(Status(StatusCode.ERROR, str(e)))
            raise
        else:
            span.set_status(Status(StatusCode.OK))
```

Manual: Go

```
import (  
    "go.opentelemetry.io/otel"  
    "go.opentelemetry.io/otel/attribute"  
    "go.opentelemetry.io/otel/codes"  
)  
  
func processOrder(ctx context.Context, orderID string) error {  
    tracer := otel.Tracer("order-service")  
    ctx, span := tracer.Start(ctx, "process-order")  
    defer span.End()  
  
    span.SetAttributes(attribute.String("order.id", orderID))  
  
    if err := validateOrder(ctx, orderID); err != nil {  
        span.RecordError(err)  
        span.SetStatus(codes.Error, err.Error())  
        return err  
    }  
  
    if err := chargePayment(ctx, orderID); err != nil {  
        span.RecordError(err)  
        span.SetStatus(codes.Error, err.Error())  
        return err  
    }  
  
    return shipOrder(ctx, orderID)  
}
```

The semi-automatic way

Annotations and decorators...

Decorators: Python 🐍

```
# This could exist but doesn't in OpenTelemetry AFAIK
from hypothetical_otel import trace

@trace.instrument("process-order")
def process_order(order_id: str):
    validate_order(order_id)
    charge_payment(order_id)
    ship_order(order_id)
```

Annotations: Java

```
import io.opentelemetry.instrumentation.annotations.WithSpan;
import io.opentelemetry.instrumentation.annotations.SpanAttribute;

public class OrderService {
    @WithSpan("process-order")
    public void processOrder(@SpanAttribute("order.id") String orderId) {
        // Clean business logic
        validateOrder(orderId);
        chargePayment(orderId);
        shipOrder(orderId);
    }
}
```

Go 🐹: no annotation support

```
// No annotation support in Go!  
// 🎉 YAY! Simplicity! 🍌  
func processOrder(ctx context.Context, orderID string) error {  
    ctx, span := tracer.Start(ctx, "process-order")  
    defer span.End()  
  
    span.SetAttributes(attribute.String("order.id", orderID))  
    // ... error handling boilerplate ...  
}  
  
// Every. Single. Function. 🤖
```

The fully automatic way

Zero code changes required...

Fully automatic: Python 🐍

```
# Install and bootstrap  
pip install opentelemetry-distro opentelemetry-exporter-otlp  
opentelemetry-bootstrap -a install  
  
# Run with zero code changes  
opentelemetry-instrument python myapp.py
```

Works automatically with: Flask, Django, FastAPI, requests, psycopg2, redis...

Fully automatic: Java

```
# Download the agent
```

```
curl -L -o agent.jar https://github.com/open-telemetry/opentelemetry-java-instrumentation/releases/latest/download/opentelemetry-javaagent.jar
```

```
# Run with zero code changes
```

```
java -javaagent:agent.jar -jar myapp.jar
```

Works automatically with: Spring, Servlet, JDBC, HTTP clients, Kafka...

Go 🐹 still manual

```
// 🎉 YAY! Simplicity! 🥳  
func processOrder(ctx context.Context, orderID string) error {  
    ctx, span := tracer.Start(ctx, "process-order")  
    defer span.End()  
  
    span.SetAttributes(attribute.String("order.id", orderID))  
    // ... error handling boilerplate ...  
}
```

No agent. No magic. Just ~~pain~~ boilerplate.

The pain is real

Manual instrumentation means

Challenge	
✗	Writing boilerplate in every function
✗	Remembering to add spans for new code
✗	Inconsistent coverage across teams
✗	Maintenance burden when requirements change



Writing boilerplate in every function



Remembering to add spans for new code



Inconsistent coverage across teams



Maintenance burden when requirements change

What Go developers want:

"The same zero-friction experience other languages have..."

✓	Goal
✓	Automatic instrumentation
✓	No runtime performance overhead
✓	No manual code changes



Automatic instrumentation



No runtime performance overhead



No manual code changes

How can Go compete with runtime magic?

Go's constraints:

Constraint	Why It Matters
✗ No annotations	Can't use <code>@WithSpan</code>
✗ No runtime code injection	Can't modify behavior at runtime
✓ Must compile instrumentation	Everything baked into binary

Go has a hidden superpower...



The toolchain is just another Go program

Go Puts its Magic in Tools, not the Language

SHOW OF HANDS 🙋

Who has ever heard about `toolexec`?

Introducing -toolexec

Go's best kept secret

```
$ go help build | grep -A6 -- -toolexec
      -toolexec 'cmd args'
```

a program to use to invoke toolchain programs like vet and asm.

For example, instead of running `asm`, the `go` command will run

```
'cmd args /path/to/asm <arguments for asm>'.
```

The TOOLExec_IMPORTPATH environment variable will be set, matching 'go list -f {{.ImportPath}}' for the package being built.

With `-toolexec`, We
Intercept Everything



The basic wrapper

```
package main

import (
    "fmt"
    "os"
    "os/exec"
    "strings"
    "time"
)

func main() {
    if len(os.Args) < 2 {
        fmt.Fprintf(os.Stderr, "TOOLExec: error: no tool specified\n")
        os.Exit(1)
    }

    tool := os.Args[1]
    args := os.Args[2:]

    fmt.Fprintf(os.Stderr, "TOOLExec: %s %s\n", tool, strings.Join(args, " "))

    start := time.Now()
    cmd := exec.Command(tool, args...)
    cmd.Stdout = os.Stdout
    cmd.Stderr = os.Stderr
    err := cmd.Run()

    duration := time.Since(start)
    fmt.Fprintf(os.Stderr, "TOOLExec: completed in %v\n", duration)

    if err != nil {
        os.Exit(1)
    }
}
```

Usage & output

```
go build -a -toolexec="$PWD/bin/toolexecwrapper" -o bin/demo-wrapped .
```

What you see (trimmed):

```
TOOLEXEC: .../compile -o $WORK/b001/_pkg_.a -trimpath $WORK/b001=> -p main -lang=go1.25 -com ...  
TOOLEXEC: completed in 11.76125ms  
TOOLEXEC: .../link -o $WORK/b001/exe/a.out -importcfg $WORK/b001/importcfg.link -X=runtime.g ...  
TOOLEXEC: completed in 41.859167ms
```

Every tool call is intercepted. And we can see what is going on.

One `go build -a`, 106 tool calls

```
57 compile
```

```
48 asm
```

```
1 link
```

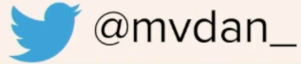
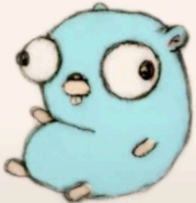
https://www.youtube.com/watch?v=sTXc_JxmvV0

Deep dive into Go's build system - Daniel Martí



Deep dive into Go's build system

GopherCon UK, 2022
Daniel Martí - cuelang.org



Daniel Martí
Deep dive into Go's build system

Target the Compiler

Focus on `compile` - that's where the source code is

```
func run() error {  
    tool := os.Args[1]  
    args := os.Args[2:]  
  
    // Only intercept compile operations  
    if strings.Contains(tool, "compile") {  
        return handleCompile(tool, args)  
    }  
  
    // Pass through other tools unchanged  
    return runTool(tool, args)  
}
```

Access the source code

Extract Go files from compiler arguments

```
func handleCompile(tool string, args []string) error {  
    // Find Go source files in args  
    var goFiles []string  
    for _, arg := range args {  
        if strings.HasSuffix(arg, ".go") {  
            goFiles = append(goFiles, arg)  
        }  
    }  
  
    // Parse and analyze each Go file  
    for _, file := range goFiles {  
        if err := analyzeGoFile(file); err != nil {  
            return err  
        }  
    }  
  
    // Continue with normal compilation  
    return runTool(tool, args)  
}
```

SHOW OF HANDS 🙋

Who has written linters for Go?

AST manipulation ✨

🔍 Parse → 🔄 Transform → 📝 Generate

- Parse any Go source into a syntax tree
- Walk and analyze every node
- Modify function declarations, statements, expressions
- Generate valid Go code from modified AST

Step 1: parse

```
import (  
    "go/ast"  
    "go/parser"  
    "go/token"  
)  
  
func analyzeGoFile(filename string) error {  
    // FileSet tracks position information across files  
    fset := token.NewFileSet()  
  
    // Parse the Go file into an AST  
    node, err := parser.ParseFile(fset, filename, nil, parser.ParseComments)  
    if err != nil {  
        return err  
    }  
  
    // node is now a *ast.File containing the entire syntax tree  
    fmt.Printf("Parsed file with %d declarations\n", len(node.Decls))  
    return nil  
}
```

Step 2: walk

```
// Walk the entire AST looking for function declarations
ast.Inspect(node, func(n ast.Node) bool {
    if fn, ok := n.(*ast.FuncDecl); ok {
        fmt.Printf("Found function: %s\n", fn.Name.Name)
        // Check if this function should be instrumented
        if hasLogComment(node, fn) {
            instrumentFunction(fn)
        }
    }
    return true // Continue traversal
})

// More powerful: astutil.Apply for modifications
astutil.Apply(node, func(cursor *astutil.Cursor) bool {
    if fn, ok := cursor.Node().(*ast.FuncDecl); ok {
        if hasLogComment(node, fn) {
            removeLogComment(node, fn) // Clean up
            injectLogging(fn)           // Transform
        }
    }
    return true
}, nil)
```

Step 2: find the directive

```
// Look for our special comment annotation
func hasLogComment(file *ast.File, fn *ast.FuncDecl) bool {
    fnPos := fn.Pos()
    for _, commentGroup := range file.Comments {
        if commentGroup.End() < fnPos {
            for _, comment := range commentGroup.List {
                if strings.Contains(comment.Text, "demo:log") {
                    return true
                }
            }
        }
    }
    return false
}
```

Step 3: build new nodes

```
// Create a timing variable at function start
startDecl := &ast.AssignStmt{
    Lhs: []ast.Expr{ast.NewIdent("start")},
    Tok: token.DEFINE,
    Rhs: []ast.Expr{
        &ast.CallExpr{
            Fun: &ast.SelectorExpr{
                X: ast.NewIdent("time"),
                Sel: ast.NewIdent("Now"),
            },
        },
    },
}

// Create entry logging statement
entryLog := &ast.ExprStmt{
    X: &ast.CallExpr{
        Fun: &ast.SelectorExpr{
            X: ast.NewIdent("slog"),
            Sel: ast.NewIdent("Info"),
        },
        Args: []ast.Expr{
            &ast.BasicLit{Kind: token.STRING, Value: `"function entry"`},
            &ast.BasicLit{Kind: token.STRING, Value: `"func"`},
            &ast.BasicLit{Kind: token.STRING, Value: fmt.Sprintf(`"%s"`, funcName)},
        },
    },
}
```

Step 3: inject them

```
// Create defer statement for exit logging
deferLog := &ast.DeferStmt{
    Call: &ast.CallExpr{
        Fun: &ast.FuncLit{
            Type: &ast.FuncType{Params: &ast.FieldList{}}},
            Body: &ast.BlockStmt{
                List: []ast.Stmt{
                    // slog.Info("function exit", "duration", time.Since(start))
                },
            },
        },
    },
}

// Inject at beginning of function
fn.Body.List = append([]ast.Stmt{startDecl, entryLog, deferLog}, fn.Body.List...)
```

Step 4: generate code

```
// Add necessary imports to the file
astutil.AddNamedImport(fset, node, "slog", "log/slog")
astutil.AddImport(fset, node, "time")

// Format the modified AST back to Go source code
var buf bytes.Buffer
if err := format.Node(&buf, fset, node); err != nil {
    return fmt.Errorf("failed to format code: %w", err)
}

// Write the generated code to a new file
outputFile := filename + ".generated"
if err := os.WriteFile(outputFile, buf.Bytes(), 0644); err != nil {
    return fmt.Errorf("failed to write output: %w", err)
}

fmt.Printf("Generated %s with logging injected\n", outputFile)
```

See it in action

Input: `main.go`

```
//demo:log  
func calculateSum(a, b int) int {  
    time.Sleep(50 * time.Millisecond)  
    return a + b  
}
```

Output: `main.go.generated`

```
func calculateSum(a, b int) int {  
    start := time.Now()  
    slog.Info("function entry", "func", "calculateSum", "a", a, "b", b)  
    defer func() {  
        slog.Info("function exit", "func", "calculateSum", "duration", time.Since(start))  
    }()  
    time.Sleep(50 * time.Millisecond)  
    return a + b  
}
```

Runtime output

```
{"time":"2026-09-24T15:07:52.505494+02:00","level":"INFO","msg":"function entry","func":"calculateSum","a":10,"b":20}  
{"time":"2026-09-24T15:07:52.557701+02:00","level":"INFO","msg":"function exit","func":"calculateSum","duration":52210375}
```

`go/ast` has a comment problem

```
func simpleOperation() {  
    start := time.Now()  
    slog.Info("function entry",  
        // Simulate work  
        "func", "simpleOperation")  
}
```

- `go/ast` keeps comments by byte offset, not attached to nodes
- Real rewriters use `github.com/dave/dst` (otelc does)

Now Go has annotations...

Would this make old Java people happy or sad?

The grand plan: compile-time transformation

Instead of runtime magic, we transform at compile-time:

- **Intercept** the build process with `-toolexec`
- **Analyze** source code before compilation
- **Transform** AST to add instrumentation
- **Compile** the modified code

Result: instrumentation compiled into your binary. No agent, no runtime attach.

One more thing...



SHOW OF HANDS 🙋

Who *doesn't* like YAML?

(Be honest!)

Aspect-oriented programming

Cross-cutting concerns made simple.

Another thing that would make old Java people happy.

What is AOP?

Concern	Examples
Observability	Tracing, metrics, logging
Security	Authentication, authorization, input validation
Reliability	Circuit breakers, retries, health checks
Performance	Caching, rate limiting, profiling

Runtime magic vs. compile time

Runtime

- **Java/C#:** Annotations + bytecode weaving
- **JavaScript:** Proxies + decorators
- **Python:** Decorators + metaclasses

Compile time

- No runtime agent
- No reflection needed
- Business code stays clean

Before: a typical Go function

```
func ProcessOrder(ctx context.Context, order Order) error {  
    // Tracing  
    ctx, span := tracer.Start(ctx, "ProcessOrder")  
    defer span.End()  
  
    // Authentication  
    if !auth.IsAuthorized(ctx, "orders:write") {  
        span.SetStatus(codes.Error, "unauthorized")  
        return ErrUnauthorized  
    }  
  
    // Logging  
    log.InfoContext(ctx, "processing order", "orderId", order.ID)  
  
    // Circuit breaker  
    err := circuitBreaker.Execute(func() error {  
        return processOrderInternal(ctx, order)  
    })  
  
    // Error handling & metrics  
    if err != nil {  
        span.SetStatus(codes.Error, err.Error())  
        metrics.Counter("orders.failed").Inc()  
        return err  
    }  
  
    return nil  
}
```

After: clean business logic

```
func ProcessOrder(ctx context.Context, order Order) error {  
    // Pure business logic!  
    return processOrderInternal(ctx, order)  
}
```

What if the transformation was data?

The same `//demo:log`, as an otelc rule

```
demo_log:
  target: main
  where:
    directive: "demo:log"
  do:
    - expand_directive:
        template: |-
            start := time.Now()
            slog.Info("function entry", "func", "{{ .FuncName }}")
            defer func() {
                slog.Info("function exit", "func", "{{ .FuncName }}", "duration", time.Since(start))
            }()
  imports:
    slog: "log/slog"
    time: "time"
```

Build it with otelc

```
otelc --rules log.otelc.yml go build -o bin/otelc-log .  
./bin/otelc-log
```

```
2026/09/24 15:07:55 INFO function entry func=calculateSum  
2026/09/24 15:07:55 INFO function exit func=calculateSum duration=52.205208ms  
Sum: 30
```



OpenTelemetry Go Compile-Time Instrumentation

otelc in one slide

- An OpenTelemetry project, built by the Go Compile-Time Instrumentation SIG
- Stable since v1.0.1 (July 2026); latest release v1.1.0 (August 2026)
- Instruments your code, its dependencies, and the standard library at build time
- No agent, nothing attached at runtime

Where it came from

Input	From
Orchestrion	Datadog
opentelemetry-go-auto-instrumentation	Alibaba
<code>instrgen</code> experience	Quesma

...it won't be Alibaba's or Datadog's solution that "wins".

Fully automatic: Go

```
# Install  
go install go.opentelemetry.io/otelc/tool/cmd/otelc@latest  
  
# Build with zero code changes  
otelc go build -o myapp .
```

Works automatically with: `net/http`, gRPC, `database/sql`, Gin, go-redis, MongoDB, Kafka, OpenAI, Anthropic, ...

Three ways to plug it in

Wrap the command

```
otelc go build -o myapp .
```

Keep go build: prepare once, then GOFLAGS

```
otelc setup
```

```
export GOFLAGS="${GOFLAGS} '-toolexec=otelc toolexec'"
```

```
go build -o myapp .
```

As a tool dependency (Go 1.24+)

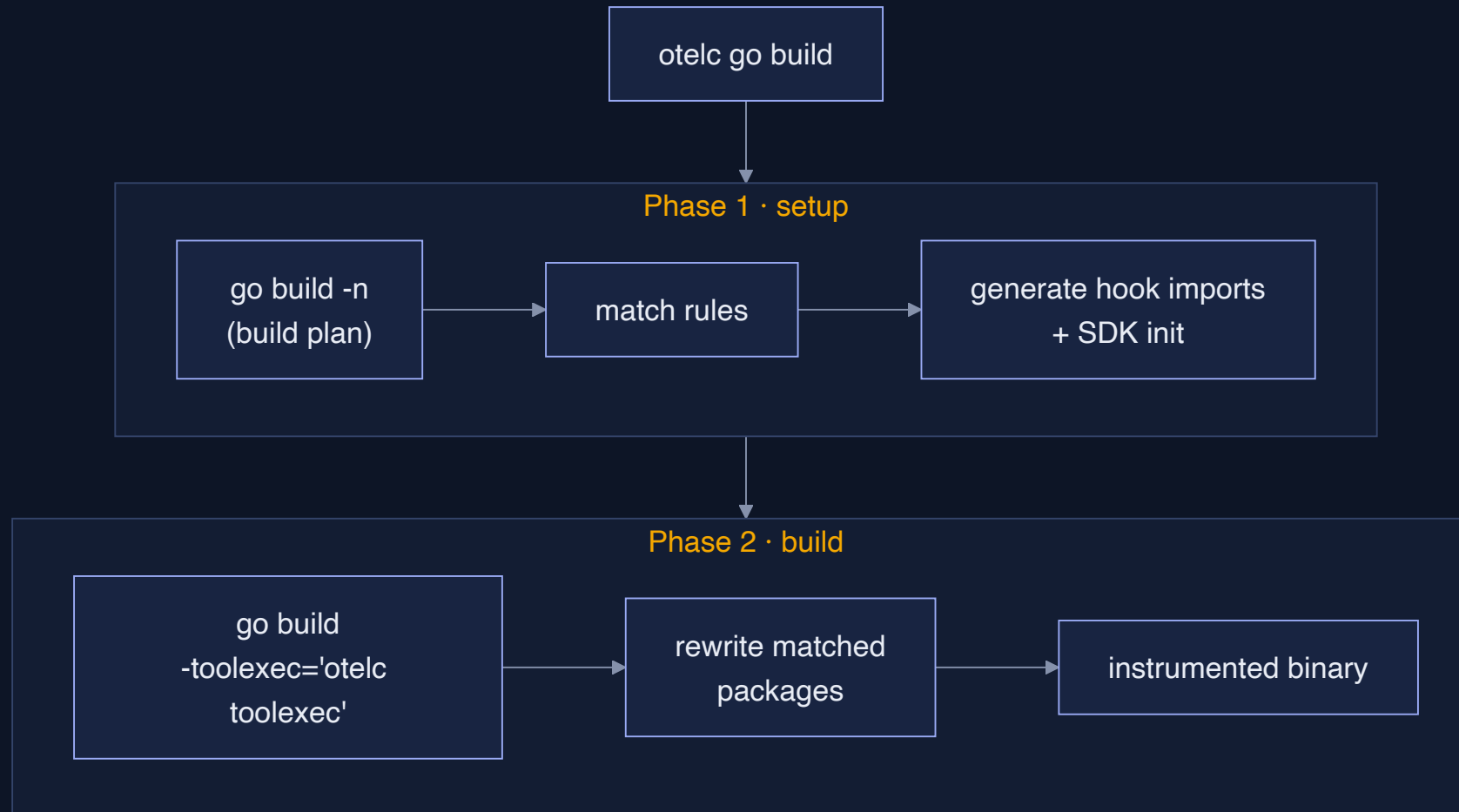
```
go get -tool go.opentelemetry.io/otelc/tool/cmd/otelc
```

```
go tool otelc go build -o myapp .
```

Under the hood



Two phases



Phase 1: setup

- Dry-run the build (`go build -n`) to list every package
- Match rules against that dependency graph
- Generate hook imports and SDK init (`otelc.runtime.go`)
- Record every match in `.otelc-build/matched.json`

What matched in a 39-line HTTP app

```
jq -r '.. | objects | .name // empty' .otelc-build/matched.json | sort -u
```

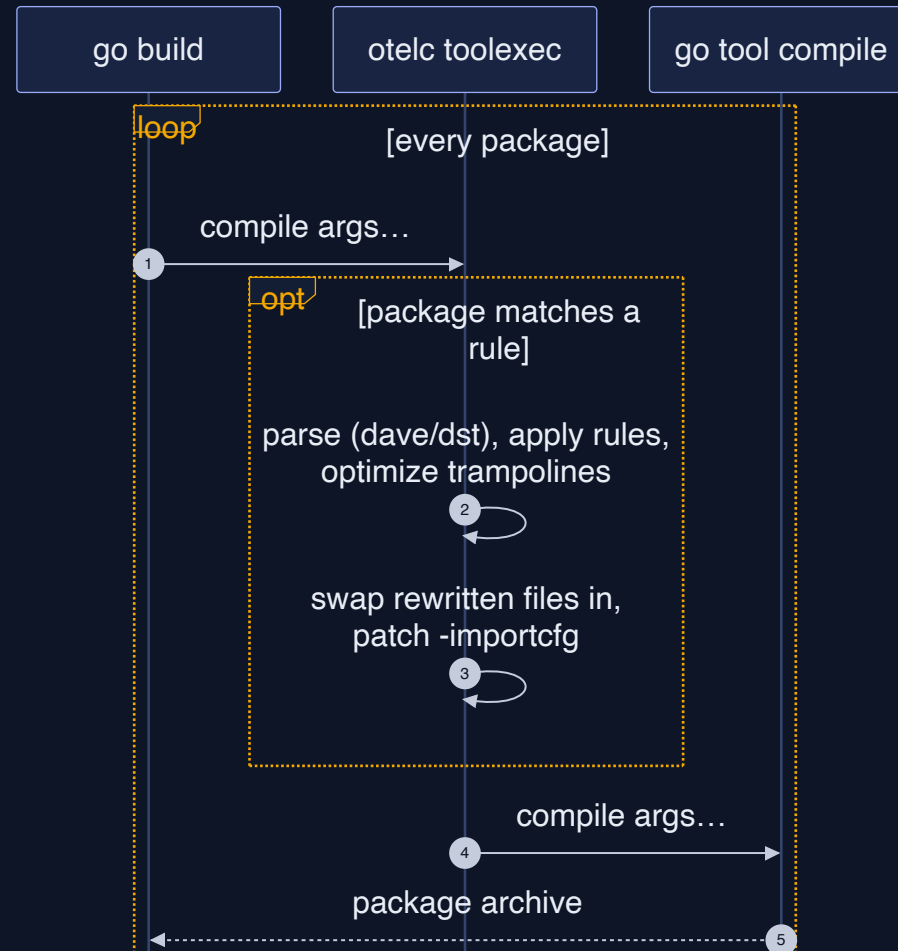
```
add_gls_field  
client_hook  
glslinker  
goroutine_propagate  
hook_log_output  
init_sdk  
otel_baggage_container  
otel_trace_context  
server_hook
```

The setup code you didn't write

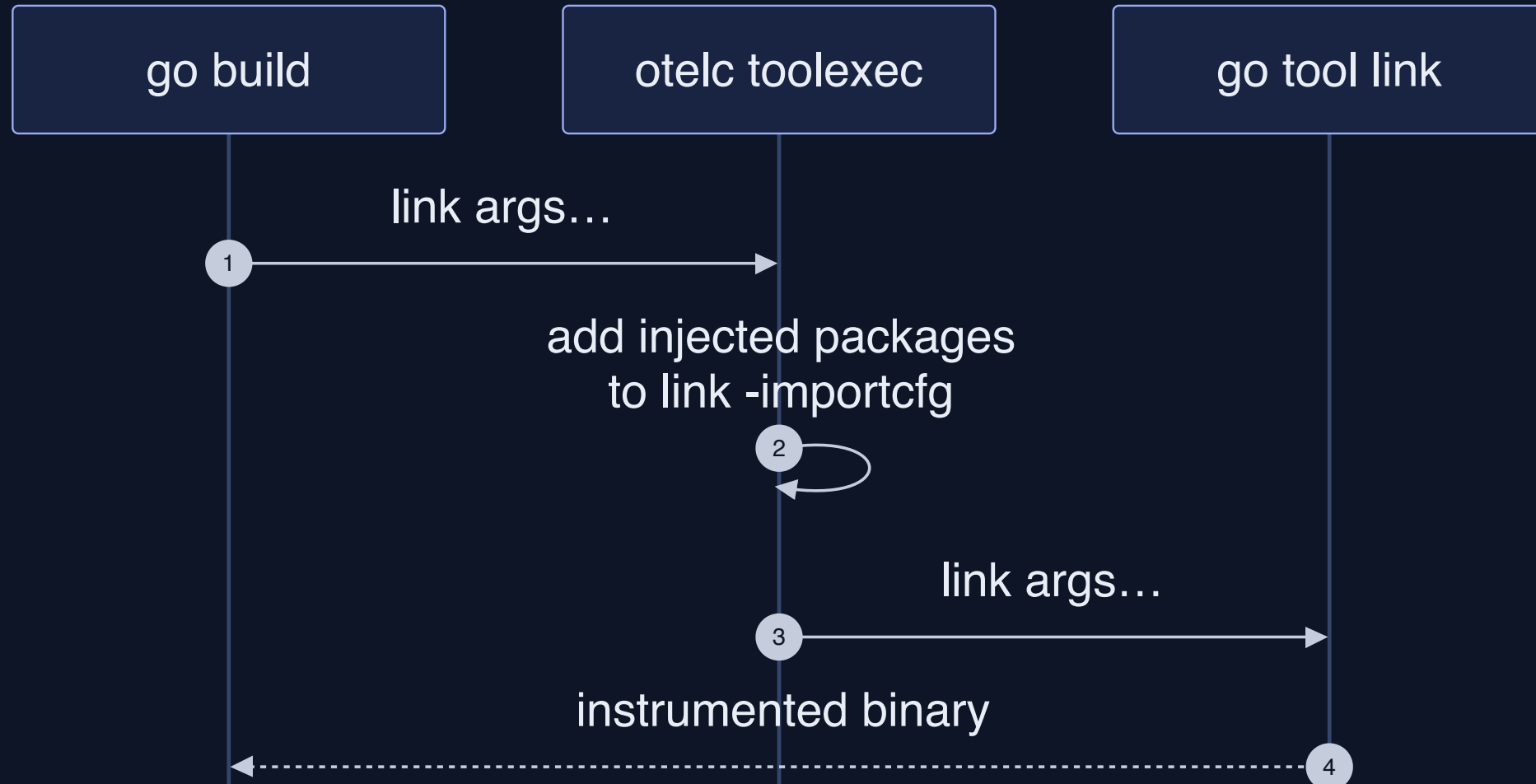
```
init_sdk:  
  target: main  
  do:  
    - add_file:  
      file: init_otelsdk.go  
      path: "go.opentelemetry.io/otelc/instrumentation/go.opentelemetry.io/otel/init"
```

```
//go:build ignore  
  
package main  
  
import (  
    "go.opentelemetry.io/otelc/pkg/runtime"  
)  
  
func init() {  
    // Initialize OpenTelemetry SDK (sets up global tracer and meter providers)  
    runtime.SetupOTelSDK()  
  
    // Start runtime metrics (respects OTEL_GO_ENABLED/DISABLED_INSTRUMENTATIONS)  
    runtime.StartRuntimeMetrics()  
}
```

Phase 2: compile



Phase 2: link



A rule: `net/http` server

```
server_hook:  
  target: net/http  
  where:  
    func: ServeHTTP  
    recv: serverHandler  
  do:  
    - inject_hooks:  
      before: BeforeServeHTTP  
      after: AfterServeHTTP  
      path: "go.opentelemetry.io/otelc/instrumentation/net/http/server"
```

A hook: BeforeServeHTTP

```
func BeforeServeHTTP(ictx hook.HookContext, recv interface{}, w http.ResponseWriter, r *http.Request) {  
    // ...  
    ctx := propagator.Extract(r.Context(), propagation.HeaderCarrier(r.Header))  
    // ...  
    ctx, span := tracer.Start(ctx,  
        spanName,  
        trace.WithSpanKind(trace.SpanKindServer),  
        trace.WithAttributes(attrs...),  
    )  
  
    // ...  
    newReq := r.WithContext(ctx)  
    ictx.SetParam(requestIndex, newReq)  
  
    // Store data for after hook  
    ictx.SetData(map[string]interface{}){  
        "ctx":    ctx,  
        "span":   span,  
        "start":  time.Now(),  
    })  
}
```

What the compiler actually sees

```
func (sh serverHandler) ServeHTTP(rw ResponseWriter, req *Request) {  
    //line <generated>:1  
    if hookContext4219161129, _ := OtelBeforeTrampoline_ServeHTTP4219161129(&sh, &rw, &req); false {  
    } else {  
        defer OtelAfterTrampoline_ServeHTTP4219161129(hookContext4219161129)  
    }  
    //line server.go:3405:2  
    handler := sh.srv.Handler  
    //line server.go:3406:2  
    if handler == nil {  
        handler = DefaultServeMux  
    }  
    // ...  
}
```

Wired with `//go:linkname`

```
//go:linkname BeforeServeHTTP go.opentelemetry.io/otelc/instrumentation/net/http/server.BeforeServeHTTP  
func BeforeServeHTTP(hookContext HookContext, recv0 interface{}, param0 ResponseWriter, param1 *Request)  
//go:linkname AfterServeHTTP go.opentelemetry.io/otelc/instrumentation/net/http/server.AfterServeHTTP  
func AfterServeHTTP(hookContext HookContext)
```

- A bodiless declaration inside the instrumented package
- The linker binds it to the hook package's function

Why `if ...; false {} else { defer ... }`?

```
// What otelc emits  
if ctx, _ := otel_trampoline_before(&arg); false {  
    ;  
} else {  
    defer otel_trampoline_after(ctx, &retval)  
    // ...  
}
```

```
// What the compiler turns it into  
ctx, _ := otel_trampoline_before(&arg)  
defer otel_trampoline_after(ctx, &retval)
```

- The `if` skeleton keeps trampolines inlinable
- Dead-code elimination and SCCP delete the empty branch

Debuggable by design

- `//line` directives map generated code back to your files and lines
- A panicking hook is recovered inside its trampoline
- Every rewritten file stays on disk: `$WORK` and `.otelc-build/`

Link: closing the loop

- Hook packages were never in your import graph
- otelc patches each compile's `-importcfg`, then the linker's
- It marks the tool identity Go hashes into cache keys, so instrumented and plain builds never collide

Runtime: plain OpenTelemetry configuration

```
OTEL_SERVICE_NAME=hello \  
OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318 \  
./hello  
  
OTEL_GO_DISABLED_INSTRUMENTATIONS=nethttp ./hello    # one off  
OTEL_SDK_DISABLED=true ./hello                       # all off
```

- Standard `OTEL_*` SDK variables; OTLP over HTTP by default

Zero lines changed, real spans

Span	Kind	Trace	Parent
GET /world	server	be645696	d0d4b59b
GET	client	be645696	4e29bac0
GET /hello	server	be645696	—

Eight rule types

Modifier	Does
<code>inject_hooks</code>	Call hooks at a function's entry and exit
<code>inject_code</code>	Prepend raw Go code to a function
<code>add_struct_fields</code>	Add fields to a struct
<code>add_file</code>	Add a Go file to a package
<code>wrap_call</code>	Wrap matching call sites
<code>expand_directive</code>	Expand a template into <code>//directive</code> functions
<code>assign_value</code>	Replace or wrap a named declaration's value
<code>set_fields</code>	Set fields on composite literals

Supported libraries (v1.1.0)

Library	Semantic Conventions
<code>net/http</code> (client & server)	HTTP spans
<code>google.golang.org/grpc</code> (client & server)	gRPC/RPC spans
<code>database/sql</code>	DB client spans
<code>github.com/gin-gonic/gin</code>	HTTP server spans
<code>github.com/redis/go-redis/v9</code>	Redis DB spans
<code>go.mongodb.org/mongo-driver</code>	MongoDB DB spans
<code>k8s.io/client-go</code>	K8s resource spans

Library	Semantic Conventions
<code>github.com/openai/openai-go</code> (v1/v2/v3)	GenAI spans
<code>github.com/anthropics/anthropic-sdk-go</code>	GenAI spans
<code>github.com/segmentio/kafka-go</code>	Kafka messaging spans
<code>github.com/aws/aws-sdk-go-v2</code>	AWS SDK client spans
<code>github.com/linode/linodego/v2</code>	HTTP client spans and metrics
<code>log</code> , <code>log/slog</code> , <code>github.com/sirupsen/logrus</code>	Trace/span ID log correlation

Go runtime metrics are on by default.

Not just tracing

The goal is to provide a flexible and non-intrusive mechanism for instrumenting Go applications, particularly for use cases such as observability (e.g., OpenTelemetry), debugging, security or performance profiling.

otelc implementation notes

Where it fits



Go instrumentation options

Approach	You change	Best when
Manual (OpenTelemetry Go API)	Your code	Custom spans and domain attributes
otelc (compile time)	Your build command	You can rebuild and want dependencies and the standard library covered
OBI (eBPF)	Nothing in the app; you deploy an agent	You can't rebuild, or run a polyglot Linux fleet

otelc and OBI, side by side

	otelc	OBI
When	Build time	Runtime (eBPF)
Languages	Go	Go, Java, .NET, Node.js, Python, Ruby, Rust, ...
Platform	Linux, macOS, Windows	Linux 5.8+ with BTF
Privileges	None	Kernel capabilities, per feature
Prebuilt binaries	No	Yes
Depth	Your code, dependencies, stdlib	Network and supported library boundaries
Status	v1 (stable)	v0 (breaking changes possible)

They compose

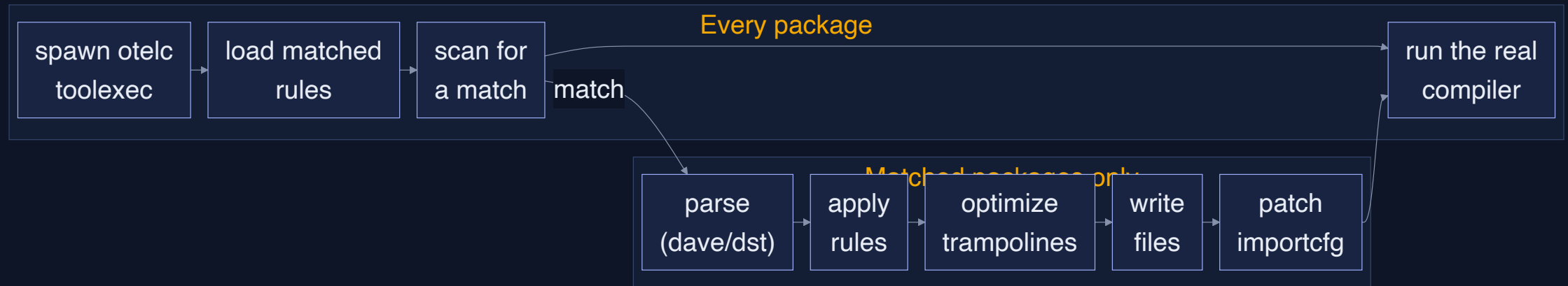
- Manual spans add domain detail on top of either
- OBI for fleet-wide boundaries; otelc where Go services need depth
- Deeper comparison posts are in progress with the OBI and Go SIGs

Adoption considerations

Before you start

- Go 1.25 or newer
- You own the build: prebuilt binaries are out of reach
- Coverage = supported libraries + your own rules
- New instrumentation ships with your next build and deploy

Where build time goes



Measure it

```
otelc --stats go build .
```

- Every package pays the pass-through; matched ones pay the rewrite
- CI fails if a benchmark scenario exceeds +150% of plain `go build`

Runtime cost

No additional overhead from the instrumentation tool itself, on top of the overhead incurred by the injected OpenTelemetry SDK code.

otelc README

- Trampolines flatten to one call and one deferred call

Your repo, your build

- otelc adds `replace` directives and runs `go mod tidy`; review the `go.mod` diff
- Pin choices in `otel.instrumentation.go` with `otelc pin`; committing it is still a local workflow (#585)
- Instrumented and plain builds never share build-cache entries

When nothing shows up

```
otelc --debug go build .           # writes .otelc-build/debug.log
jq -r '.. | objects | .name // empty' .otelc-build/matched.json
OTEL_LOG_LEVEL=debug ./myapp       # surface exporter errors
otelc cleanup                       # start from a clean state
```

Known limits

- Generic targets: hooks can't `SetParam` or `SetReturnValue`
- Hook code can't import freely: no import cycles into the target
- Goroutine-local context doesn't flow back from child goroutines
- Built on toolchain internals (`-toolexec`, `go:linkname`) that can change

What's next



The roadmap

- More instrumentations across popular Go libraries
- Registry-based discovery via the OpenTelemetry Registry
- Lower build-time and runtime cost
- Docs, examples, and outreach

Try it tonight

```
go install go.opentelemetry.io/otelc/tool/cmd/otelc@latest
otelc go build -o myapp .
OTEL_SERVICE_NAME=myapp ./myapp
```

Then tell us what broke: [GitHub Discussions](#) · [Issues](#)

Instrument a library you use

- A rule (`*.otelc.yml`) says where
- A hook package (Go) says what
- Tests prove it

[Instrumentation guide](#) · [Rules reference](#)

Join the SIG

- **Meetings:** Thursdays · 09:30 UTC (11:30 CEST) · invites via [calendar-go](#)
- **Slack:** `#otel-go-compile-instrumentation` on CNCF Slack
- **Code:** [open-telemetry/opentelemetry-go-compile-instrumentation](#)
- **Calendar and notes:** [open-telemetry/community](#)

Built by

- Dario Castañé (Datadog)
- Haibin Zhang (Alibaba)
- Huxing Zhang (Alibaba)
- Kemal Akkoyun (Datadog)
- Przemyslaw Delewski (Quesma)
- Xabier Martinez (Cabify)
- Yi Yang (Alibaba)
- Approver: Azhar Momin (joined through LFX Mentorship)

...and every contributor

Key takeaways

- `-toolexec` makes the Go toolchain the hook Go never had
- Compile-time rewrites stay readable: every rewritten file is on disk
- otelc is the OpenTelemetry way to do it: v1, vendor-neutral, built by a SIG

Thank you · Questions?

- GitHub: [@kakkoyun](#)
- Blog: [kakkoyun.me](#)
- Bluesky: [kakkoyun.me](#)
- LinkedIn: [kakkoyun](#)
- X: [@kakkoyun_me](#)



Slides, demos, captures:

github.com/kakkoyun/otel-night-berlin-2026

Up next



Juraci Paixão Kröhling

Instrumenting Go with coding agents and the `otel-go` skill

Bonus: dark magic

otelc patches the Go runtime

```
add_gls_field:
  target: "runtime"
  where:
    struct: "g"
  do:
    - add_struct_fields:
        new_field:
          - name: "otel_trace_context"
            type: "interface{}"
          - name: "otel_baggage_container"
            type: "interface{}"
```

...adds functions to package `runtime`

```
func GetTraceContextFromGLS() interface{} {  
    return getg().m.curg.otel_trace_context  
}  
  
func GetBaggageContainerFromGLS() interface{} {  
    return getg().m.curg.otel_baggage_container  
}  
  
func SetTraceContextToGLS(traceContext interface{}) {  
    getg().m.curg.otel_trace_context = traceContext  
}  
  
func SetBaggageContainerToGLS(baggageContainer interface{}) {  
    getg().m.curg.otel_baggage_container = baggageContainer  
}
```

...and copies context into every new goroutine

```
goroutine_propagate:  
  target: "runtime"  
  where:  
    func: "newproc1"  
  do:  
    - inject_code:  
      raw: |  
        defer func(){  
          _unnamedRetVal0.otel_trace_context = propagateOtelContext(callergp.otel_trace_context);  
          _unnamedRetVal0.otel_baggage_container = propagateOtelContext(callergp.otel_baggage_container);  
        }()
```

Calling a function Go doesn't have

```
ctx := &traceContext{
    traceID: "123",
    spanID:  "456",
}
runtime.SetTraceContextToGLS(ctx)

go func() {
    fmt.Printf("traceContext from parent goroutine: %s\n", runtime.GetTraceContextFromGLS())
}()
```

`runtime` is the standard library; these exist only after otelc runs.

The Go team has a name for this

```
// gopark should be an internal detail,  
// but widely used packages access it using linkname.  
// Notable members of the hall of shame include:
```

- `go:linkname` variables resolve by symbol load order ([golang/go#72032](https://golang.org/issue/72032))